

AULA 02 • FRONT-END FRAMEWORKS

JavaScript moderno I: funções e dados

Construindo um painel de perfis de música, jogos e filmes

Prof. Diogo Rodrigues

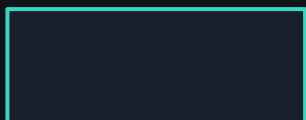
Roteiro



Valores, funções e o diagrama entrada → função → saída



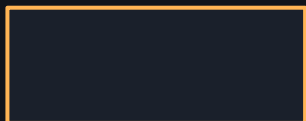
Previsões de saída + demonstração ao vivo



Template literals, objetos e destructuring



Prática em trios: painel de perfis



Atualização imutável com spread + demonstração



Erros frequentes e síntese rumo a componentes

const e let: o que pode mudar?

```
const NOME_APP = 'Painel de Perfis';  
let contadorDeCurtidas = 0;  
  
contadorDeCurtidas = contadorDeCurtidas +  
1; // ok  
NOME_APP = 'Outro'; // TypeError
```

Regra prática

Comece sempre com const.
Troque para let só quando precisar reatribuir.

Atenção

const trava o nome, não o conteúdo do objeto.

Entrada → função → saída



A função é uma caixa: mesma entrada, mesma saída. Ela não precisa saber quem a chamou.

Função declarada

```
function saudacao(nome) {  
  return 'Olá, ' + nome + '!';  
}  
  
saudacao('Ana'); // 'Olá, Ana!'
```

- Tem nome próprio: aparece bem nas mensagens de erro
- Sofre **hoisting**: pode ser chamada antes da linha em que foi escrita
- Boa para funções principais do arquivo

Arrow function

```
const saudacao = (nome) => {  
  return 'Olá, ' + nome + '!';  
};  
  
// return implícito, uma linha só:  
const dobro = (n) => n * 2;
```

Sem chaves = return

dobro(4) devolve 8 sem escrever return.

Com chaves = precisa return

Esqueceu o return dentro de {}? A função devolve undefined.

Parâmetros: o contrato da função

```
function curtir(perfil, quantidade = 1) {  
  return perfil + ' +' + quantidade;  
}
```

```
curtir('Ana');           // 'Ana +1'  
curtir('Ana', 5);        // 'Ana +5'  
curtir();                // 'undefined +1'
```

- Parâmetro é o nome no molde; argumento é o valor real
- Valor padrão entra quando o argumento é undefined
- Ordem importa: trocar a ordem troca o resultado

O que aparece no console?

```
function somar(a, b) {  
  console.log(a + b);  
}
```

```
const resultado = somar(2, 3);  
console.log(resultado);
```

return ≠ console.log

return

Devolve o valor para quem chamou. Dá para guardar, somar, passar adiante.

console.log

Só mostra na tela. Some quando o app roda de verdade.

```
function somar(a, b) {  
  return a + b;  
}  
  
const total = somar(2, 3); // 5  
console.log(total * 10);   // 50
```

Ao vivo: montando o cartão de perfil

- Escrever `formatarPerfil()` do zero, com erro proposital
- Ler a mensagem de erro em voz alta antes de corrigir
- Trocar `console.log` por `return` e ver a diferença

Template literals

```
const nome = 'Ana';  
const jogo = 'Celeste';  
  
// antes  
' ' + nome + ' joga ' + jogo  
  
// agora, com crases  
`${nome} joga ${jogo}`
```

- Crase (``) abre e fecha, não aspas
- \${ } aceita qualquer expressão:
\${nota * 2}
- Quebra de linha real dentro do texto

Qual texto sai daqui?

```
const usuario = 'Bia';  
const nota = 4;  
  
const frase = `${usuario} deu ${nota + 1}  
estrelas`;  
console.log(frase);
```

Cuidado

Onde entra o cálculo?
Antes ou depois de virar
texto?

Objetos: o perfil do painel

```
const perfil = {  
  nome: 'Ana',  
  musica: 'Djavan',  
  jogo: 'Celeste',  
  filme: 'Bacurau',  
  curtidas: 12,  
};  
  
perfil.jogo;      // 'Celeste'  
perfil['filme'];  // 'Bacurau'
```

Chave e valor

Chave é o rótulo; valor é o conteúdo.
Acesse com ponto.

const não congela

perfil.curtidas = 13 funciona mesmo com const. Trocar o perfil inteiro é que não.

Destructuring: abrir o pacote

```
const { nome, jogo } = perfil;

// direto no parâmetro:
function cartao({ nome, musica, curtidas }) {
  return `${nome} · ${musica} · ${curtidas}`;
}

cartao(perfil); // 'Ana · Djavan · 12'
```

- O nome da variável precisa bater com a chave
- Chave inexistente vira undefined
- Deixa a assinatura da função autoexplicativa

Painel de perfis: mãos no código

- 1 Criem 3 perfis (nome, musica, jogo, filme, curtidas)
- 2 `cartao(perfil)` devolve uma frase com template literal
- 3 `resumo(perfis)` devolve quantos perfis existem no total
- 4 `curtir(perfil)` devolve um NOVO perfil com +1 curtida
- 5 Nada de `console.log` dentro das funções — só return

Como saber que ficou bom

Funciona

As quatro funções devolvem valores; nada de undefined inesperado.

Legível

Nomes dizem o que a função faz. Sem f1, x, temp.

Sem efeito colateral

curtir() não altera o perfil original.

Entrega

Um arquivo por trio + os 3 nomes no topo em comentário.

Rodízio comprovado

Cada integrante digitou pelo menos um bloco de código.

O perfil original mudou?

```
const perfil = { nome: 'Ana', curtidas: 12 };

function curtir(p) {
  p.curtidas = p.curtidas + 1;
  return p;
}

const novo = curtir(perfil);
console.log(perfil.curtidas, novo.curtidas);
```

Pense

Quantos objetos
existem na memória
depois disso?

Spread: copiar e trocar um campo

```
function curtir(perfil) {  
  const n = perfil.curtidas + 1;  
  return { ...perfil, curtidas: n };  
}  
  
const novo = curtir(perfil);  
perfil.curtidas; // 12  (intacto)  
novo.curtidas;   // 13  (novo objeto)
```

- ...perfil copia todos os campos
- O campo repetido depois vence
- A ordem importa: sobrescrita fica por último

Cópia rasa: hoje só perfis com valores simples. Objeto dentro de objeto fica para a próxima aula.

Prevejam e depois assistam

```
const a = { tema: 'claro', fonte: 14 };  
const b = { ...a, fonte: 18 };  
console.log(a.fonte, b.fonte, a === b);
```

Escrevam os três valores.

- Refatorar `curtir()` ao vivo para versão imutável
- Aplicar em uma lista de perfis com `.map()`
- Mostrar o painel inteiro renderizado no console

Erros frequentes e o que vem depois

Cinco tropeços do dia

- Esquecer o return dentro de { }
- Tratar console.log como retorno
- Aspas em vez de crase no template
- Alterar o objeto original sem querer
- Destructuring com chave errada

Síntese

- Função = caixa: entrada → saída
- Dados moram em objetos
- Destructuring abre o pacote
- Atualizar = criar objeto novo